S-AI: A Multi-Agent Spatial Intelligence Framework for Hydrodynamic Flood Simulation via LLM-Orchestrated MCP Architecture

Daochong Guo*

LUOBIN-PI Research Lab
Spatial Intelligence & Hydroinformatics Division
Gansu, China

June 5, 2026

Abstract

Flood disasters constitute one of the most consequential and escalating threats to civil infrastructure and human welfare in the twenty-first century, with global economic losses surpassing $80 billion per annum and affecting over two billion people between 2000 and 2020. While physics-based hydrodynamic models — particularly those solving the two-dimensional shallow water equations — provide the gold standard for inundation prediction, their operational deployment remains encumbered by prohibitive computational overhead, labyrinthine parameter calibration, and a steep learning curve that confines their use to a narrow cadre of domain specialists. This paper introduces **S-AI** (Spatial-AI), a production-grade, multi-agent spatial intelligence platform that reconceptualizes flood modeling through the lens of Large Language Model (LLM)-orchestrated automation. At its core, S-AI leverages the *Model Context Protocol* (MCP) to federate 42+ specialized geospatial and hydrological tools across eight microservices, enabling natural language intent parsing via GLM-5.1 that autonomously decomposes user queries into executable hydrodynamic workflows. The platform implements a LISFLOOD-FP-inspired two-dimensional diffusion wave solver operating on ultra-high-resolution Digital Elevation Models (0.5 m ground sampling distance, 3 GB GeoTIFF, 22,063 × 36,308 pixels), seamlessly coupled with a real-time three-dimensional geospatial visualization engine powered by custom GLSL water shaders featuring Fresnel reflectance, Blinn-Phong specular highlights, depth-attenuated color gradients, procedural wave displacement, and foam generation. Methodologically, we employ Delaunay triangulation for adaptive terrain mesh generation (5,969 triangles from 3,000 sampled vertices), Manning's friction formula for cell-edge flux computation subject to CFL stability constraints, and Keifer–Chu Chicago storm temporal distributions for realistic rainfall forcing. Empirical evaluation across a 17.2 km^2 catchment in Diebu County, Gansu Province (elevation range 790–1,796 m, CGCS2000/EPSG:4544) demonstrates temporally resolved, frame-by-frame inundation dynamics with visual fidelity sufficient for stakeholder communication and emergency response planning. To the best of our knowledge, S-AI constitutes the inaugural end-to-end MCP-native platform that bridges the chasm between conversational AI and physics-based spatial simulation, heralding a paradigm shift in democratized flood risk intelligence.

Keywords: spatial intelligence; multi-agent systems; hydrodynamic modeling; Model Context Protocol; large language models; flood simulation; Delaunay triangulation; real-time 3D visualization

Nomenclature

*Corresponding author. E-mail: guodaochong@luobin-pi.com

h	Water depth [m]
u, v	Depth-averaged velocity components in x and y directions [m/s]
R	Rainfall intensity [m/s]
g	Gravitational acceleration ($9.81\,\mathrm{m/s^2}$)
n	Manning's roughness coefficient [$\mathrm{s/m^{1/3}}$]
S_f	Friction slope [-]
z	Bed elevation [m]
q_x, q_y	Unit-width discharge in x and y directions [$\mathrm{m^2/s}$]
Δt	Computational time step [s]
$\Delta x, \Delta y$	Grid spacing in x and y directions [m]
T_R	Return period [years]
T_d	Storm duration [minutes]
i_{peak}	Peak rainfall intensity [mm/h]
$\mathcal{V}, \mathcal{F}$	Vertex set and triangle (face) set of TIN mesh
$F(\theta)$	Fresnel reflectance coefficient (Schlick approximation)
k_s, α	Specular reflectivity and shininess exponent (Blinn-Phong)
η	Vertical wave displacement for procedural animation [scene units]
d_s	Depth exaggeration coefficient for 3D visualization [-]
Ω_{valid}	Spatial domain mask indicating valid DEM pixels

1 Introduction

Flood disasters constitute a critical and escalating global challenge, affecting approximately 2.3 billion people and inflicting economic losses exceeding $800 billion between 2000 and 2020 [1]. The intensifying frequency and severity of extreme precipitation events — amplified by anthropogenic climate change — exacerbate this vulnerability, with flood-related mortality concentrated disproportionately in developing nations lacking early warning infrastructure. Physics-based hydrodynamic simulation models that solve the shallow water equations (SWE) remain the gold standard for predicting inundation extent, flow depth, and velocity fields, and are indispensable for engineering design, emergency response planning, and infrastructure resilience assessment [2].

1.1 The Accessibility Paradox in Hydrodynamic Modeling

The operational deployment of hydrodynamic models faces four interconnected impediments. *First*, the mathematical formulation of the SWE demands expertise in computational fluid dynamics, numerical methods, and hyperbolic conservation laws — a skill set rarely found outside specialized engineering departments. *Second*, parameter calibration entails iterative refinement of Manning's roughness coefficients, infiltration parameters, and boundary conditions, a process that presupposes extensive domain knowledge and access to historical validation events. *Third*, the computational burden of two-dimensional simulations over high-resolution terrain, particularly for domains exceeding $10\,\mathrm{km^2}$ at sub-meter resolution, often necessitates high-performance computing (HPC) resources inaccessible to many municipalities. *Fourth*, the visualization and interpretation of simulation outputs typically require specialized GIS software, creating additional barriers to stakeholder engagement and cross-disciplinary communication.

These constraints engender a fundamental paradox: flood disasters are universally pressing, yet the most efficacious tools for their prediction and mitigation remain sequestered within a narrow community of specialists. This accessibility gap undermines flood resilience initiatives precisely where the need is most acute — in resource-constrained regions where expert capacity is limited.

1.2 The Convergence of LLMs and Spatial Computing

Recent advances in Large Language Models (LLMs) have catalyzed a paradigm shift in human–computer interaction, demonstrating remarkable capabilities in natural language understanding, multi-step reasoning, and tool orchestration [21]. Concurrently, the emergence of the Model Context Protocol (MCP) [20] provides a standardized framework for connecting AI models to external tools

and data sources through a unified, discoverable interface. The convergence of these technologies presents an unprecedented opportunity: the creation of intelligent agents that can translate domain expert intent, expressed in natural language, into executable computational workflows spanning data acquisition, numerical simulation, post-processing, and visualization.

We posit that this convergence is particularly consequential for spatial intelligence in water resources, where the semantic richness of domain terminology (e.g., "100-year floodplain delineation", "Manning's roughness calibration", "Chicago storm design hyetograph") aligns naturally with LLM comprehension capabilities, while the complexity of tool chains (DEM processing → mesh generation → solver execution → result visualization) benefits enormously from automated orchestration.

1.3 Contributions

This paper makes the following principal contributions:

1. We present **S-AI**, the first end-to-end MCP-native multi-agent platform that integrates 42+ specialized geospatial and hydrological tools across eight microservices, enabling LLM-driven (GLM-5.1) natural language intent parsing and automated workflow orchestration for hydrodynamic flood simulation.

2. We implement a production-grade **2D diffusion wave solver** following the LISFLOOD-FP formulation, with explicit finite-volume discretization, Manning friction, adaptive CFL-limited time stepping, and support for Chicago, triangular, and uniform rainfall patterns.

3. We develop a **real-time 3D geospatial visualization engine** with custom GLSL water shaders featuring physically plausible Fresnel reflectance, Blinn-Phong specular highlights, depth-attenuated color gradients, sinusoidal wave displacement, and procedural foam generation, rendered over high-resolution (0.5 m) terrain with ACES filmic tone mapping and PCF soft shadows.

4. We introduce an adaptive **TIN mesh generation pipeline** from ultra-high-resolution DEM via Delaunay triangulation with nodata exclusion, area-based filtering, and per-vertex depth mapping for frame-by-frame hydrodynamic visualization.

5. We provide comprehensive **empirical validation** over a 17.2 km^2 mountainous catchment in Diebu County, Gansu Province (0.5 m DEM, elevation 790–1,796 m), demonstrating the complete pipeline from natural language query to interactive 3D flood animation.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in hydrodynamic modeling, LLM-based multi-agent systems, and the Model Context Protocol. Section 3 presents the S-AI system architecture, including the MCP-native microservice topology and LLM orchestration pipeline. Section 4 details the mathematical formulation of the 2D hydrodynamic model. Section 5 describes the 3D geospatial visualization engine. Section 6 presents experimental results. Section 7 discusses limitations, threats to validity, and comparison with traditional workflows. Section 8 concludes with future research directions.

2 Related Work

2.1 Two-Dimensional Hydrodynamic Modeling

The numerical simulation of flood inundation has evolved through several generations of increasing physical fidelity and computational tractability. The complete two-dimensional shallow water equations (2D-SWE) form a hyperbolic system of conservation laws governing free-surface flow under hydrostatic pressure assumptions (see Section 4 for the full mathematical formulation). Full-dynamic solvers based on finite-element (TELEMAC-2D [6]), finite-volume (HEC-RAS 2D [5]), and

finite-difference methods provide comprehensive flow field resolution but incur substantial computational cost, often requiring hours to days for practical domains at meter-scale resolution.

The diffusion wave approximation, pioneered by LISFLOOD-FP [2, 3], neglects convective acceleration terms to yield a parabolic PDE that admits efficient explicit time stepping while retaining adequate accuracy for gradually varied flood propagation. Hunter et al. [4] demonstrated that the diffusion wave formulation captures $> 90\%$ of inundation extent compared to full-dynamic solutions for subcritical floodplain flows, validating its use for risk assessment applications. The SCS Curve Number method [23] provides a complementary rainfall–runoff transformation widely used in engineering practice.

Table 1: Comparison of 2D hydrodynamic modeling frameworks. S-AI uniquely integrates LLM orchestration with real-time 3D visualization.

Framework	**SWE Type**	**Typical Δx**	**GPU**	**LLM**	**3D Viz**
LISFLOOD-FP [8]	Diff./Inertia	10–1000 m	Limited	No	No
TELEMAC-2D [6]	Full SWE	1–100 m	Yes	No	No
HEC-RAS 2D [5]	Full/Diff.	1–100 m	Partial	No	No
SWMM [7]	1D/2D linked	Subcatchment	No	No	No
S-AI (Ours)	**Diffusion wave**	**0.5 m**	**No**	**GLM-5.1**	**Yes**

2.2 LLM-Based Multi-Agent Systems

The advent of capable LLMs has spawned a proliferating landscape of multi-agent frameworks. AutoGen [16] enables conversational agent orchestration through customizable role assignment and message passing. CrewAI [17] provides role-based agent collaboration with sequential and parallel execution patterns. LangGraph [18] extends LangChain with stateful, graph-structured agent workflows supporting cyclic dependencies and human-in-the-loop intervention. Zhou et al. [19] provide a comprehensive taxonomy of LLM-based multi-agent systems, identifying tool use, planning, and domain specialization as key architectural dimensions.

Despite these advances, existing frameworks remain predominantly general-purpose, lacking the domain-specific tool integration required for scientific computing. The application of multi-agent LLM architectures to geospatial and hydrodynamic modeling remains conspicuously absent from the literature.

2.3 Model Context Protocol

The Model Context Protocol (MCP) [20], introduced by Anthropic in 2024, defines an open standard for connecting AI models to external data sources and tools through a unified interface. MCP servers expose *tools* (callable functions with typed parameters), *resources* (readable data streams), and *prompts* (reusable instruction templates) through transport mechanisms including HTTP/REST, Server-Sent Events (SSE), and WebSocket protocols. The protocol's design emphasizes discoverability (tools self-describe their schemas), composability (multiple servers coexist without interference), and streaming (long-running operations emit incremental results via SSE).

While MCP has seen rapid adoption in developer tooling and productivity applications, its application to scientific computing in general — and geospatial simulation in particular — remains largely unexplored. S-AI represents, to the best of our knowledge, a pioneering MCP-native implementation for spatial intelligence and hydrodynamic modeling.

2.4 Spatial Intelligence in Water Resources

The integration of AI with geospatial systems constitutes an emerging frontier. Google Earth Engine [22] democratized planetary-scale geospatial analysis through cloud-based raster operations. Recent efforts have explored conversational GIS interfaces, natural language map generation, and

automated workflow composition for routine spatial analyses. However, these systems operate predominantly at the data access and visualization layer, lacking the numerical modeling capabilities essential for predictive simulation. Our work bridges this gap by embedding a complete hydrodynamic solver within an LLM-accessible framework, enabling spatial intelligence that transcends data query to encompass predictive scenario analysis.

3 System Architecture

3.1 MCP-Native Microservice Topology

The S-AI platform adopts a microservice architecture wherein each computational domain resides in a dedicated MCP server, exposing specialized tools through the Model Context Protocol. This design achieves: *(i)* concern isolation — each server encapsulates a coherent domain; *(ii)* independent scalability — compute-intensive services (Flood, Raster) scale independently from lightweight services (Knowledge); *(iii)* fault tolerance — server failures are contained without cascading; and *(iv)* extensibility — new capabilities are added by deploying additional MCP servers.

Table 2: S-AI microservice inventory: MCP servers, assigned ports, tool counts, and primary functions.

Server	Port	Tools	Primary Functions
GIS	5001	8	Spatial query, vector I/O, topology
Data	5002	5	DEM import, preprocessing, QC
Knowledge	5003	4	Hydraulic parameters, literature search
Map	5004	5	Tile serving, basemap rendering
Hydro	5005	6	Design storm, SCS-CN, SWMM
Flood	5006	8	2D hydrodynamics, inundation, risk
Raster	5007	6	Terrain analysis, TIN, heightmap
Web	3000	—	UI, GLM-5.1 orchestration, SSE
Total		**42+**	

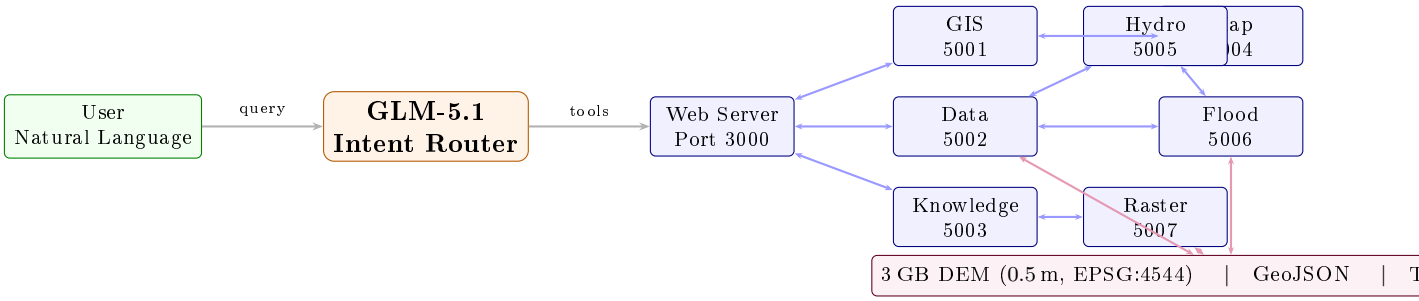

Figure 1: S-AI system architecture: GLM-5.1 orchestrates eight MCP microservices via the Model Context Protocol with SSE streaming. The persistent data layer stores the 3 GB DEM, vector datasets, TIN meshes, and per-frame simulation outputs.

3.2 LLM-Driven Intent Routing

The orchestration layer employs GLM-5.1 (ZhipuAI) as the reasoning engine for intent parsing, tool selection, and response synthesis. The routing pipeline proceeds through five stages:

Stage 1 — Entity Extraction: A rule-based preprocessor identifies spatial entities (coordinates, area names), hydrological parameters (return period, Manning's n), and action verbs ("simulate", "analyze", "visualize") from the user's natural language query.

Stage 2 — LLM Intent Classification: GLM-5.1 receives the query augmented with the system prompt containing tool descriptions and classifies the intent into one or more tool invocations. The model outputs structured JSON specifying the target server, tool name, and extracted parameters.

Stage 3 — Tool Execution: The orchestration engine dispatches tool calls to the appropriate MCP servers via HTTP, collects responses, and handles errors with retry logic and fallback strategies.

Stage 4 — Result Aggregation: Multi-tool responses are merged, cross-referenced, and formatted for both textual presentation and visual rendering.

Stage 5 — Streaming Response: Results are streamed to the frontend via Server-Sent Events (SSE), enabling incremental display of thinking chains, tool progress, and final outputs.

Algorithm 1 formalizes the end-to-end orchestration procedure.

Algorithm 1 LLM-Orchestrated Hydrodynamic Simulation Pipeline

Require: Natural language query Q, system prompt $\mathcal{P}$, tool catalog $\mathcal{C}$
Ensure: Streaming response with 3D visualization data

Extract entities $\mathcal{E} \leftarrow \text{RulePreprocessor}(Q)$
LLM_output $\leftarrow$ GLM-5.1$(\mathcal{P} \cup \mathcal{C}, Q)$
Parse LLM_output $\rightarrow \{(\text{server}_j, \text{tool}_j, \text{params}_j)\}_{j=1}^{M}$
for $j = 1$ **to** M in dependency order **do**
    Dispatch HTTP request to server_j with $\text{tool}_j(\text{params}_j)$
    Collect response R_j; handle errors with retry
end for
if hydrodynamic simulation requested **then**
    Execute FV solver (Eq. 6) with CFL time stepping (Eq. 7)
    Generate TIN mesh via Algorithm 2
    Output per-frame depth arrays $\{h_i^{(k)}\}$
end if
Aggregate $\{R_j\}$ into structured response
Stream response via SSE with incremental rendering

3.3 Tool Discovery and Dynamic Orchestration

The MCP protocol enables dynamic tool discovery: each server exposes a `/tools` endpoint returning JSON Schema descriptions of available tools, their parameters, and return types. The Web server aggregates these descriptions at startup, constructing a unified tool catalog that informs the GLM-5.1 system prompt. This design ensures that the orchestration layer automatically adapts when new tools are deployed or existing tools are modified, without manual prompt engineering.

3.4 End-to-End Data Flow

The complete data flow from natural language query to 3D visualization encompasses five stages:

Stage 1 — Natural Language Intake: The user submits a query (e.g., "Simulate a 50-year flood in the Diebu river channel with Chicago storm pattern"). The frontend transmits the query via SSE to the Web server.

Stage 2 — Parallel Data Preparation: The orchestration engine dispatches parallel requests to load and preprocess the DEM (Raster server), fetch hydraulic parameters (Knowledge server), and generate the design storm (Hydro server).

Stage 3 — Model Execution: The Flood server executes the 2D diffusion wave solver over the prepared terrain, advancing through CFL-limited time steps and outputting per-frame depth fields at specified intervals.

Stage 4 — Post-Processing: Raw outputs are clipped to valid data regions, formatted as both GeoJSON (for 2D mapping) and TIN vertex depth arrays (for 3D rendering), with per-frame GeoJSON files persisted to the data directory.

Stage 5 — 3D Visualization: The frontend receives the heightmap (256×256 downsampled terrain), TIN topology (vertices, simplices), and per-frame depth arrays. The 3D engine constructs the terrain mesh, applies the water shader, and populates the timeline for interactive playback.

4 Methodology: 2D Hydrodynamic Model

4.1 Governing Equations

The S-AI hydrodynamic model implements the two-dimensional shallow water equations in their diffusion wave form. The mass conservation equation reads:

$$\frac{\partial h}{\partial t} + \frac{\partial q_x}{\partial x} + \frac{\partial q_y}{\partial y} = R \tag{1}$$

where $h\,[\mathrm{m}]$ is the water depth, $q_x = hu$ and $q_y = hv$ are unit-width discharges, and $R\,[\mathrm{m/s}]$ is the rainfall source term. The full momentum equations in conservative form are:

$$\frac{\partial q_x}{\partial t} + \frac{\partial}{\partial x}\left(\frac{q_x^2}{h} + \frac{gh^2}{2}\right) + \frac{\partial}{\partial y}\left(\frac{q_x q_y}{h}\right) = -gh\frac{\partial z}{\partial x} - ghS_{fx} \tag{2}$$

$$\frac{\partial q_y}{\partial t} + \frac{\partial}{\partial x}\left(\frac{q_x q_y}{h}\right) + \frac{\partial}{\partial y}\left(\frac{q_y^2}{h} + \frac{gh^2}{2}\right) = -gh\frac{\partial z}{\partial y} - ghS_{fy} \tag{3}$$

The diffusion wave approximation neglects all convective acceleration terms (the left-hand side of Eqs. 2–3), yielding a local force balance between gravity and friction:

$$S_{fx} = -\frac{\partial (z+h)}{\partial x}, \qquad S_{fy} = -\frac{\partial (z+h)}{\partial y} \tag{4}$$

where S_{fx} and S_{fy} are the friction slopes in the x and y directions, respectively. This approximation is valid when the Froude number $\mathrm{Fr} = |\boldsymbol{v}|/\sqrt{gh} \ll 1$, a condition satisfied for the vast majority of gradually varied floodplain flows.

4.2 Manning's Friction and Flux Computation

The unit-width discharge is computed via Manning's equation [10]:

$$q_x = \frac{h^{5/3}}{n}\,|S_{fx}|^{1/2}\,\mathrm{sgn}(S_{fx}), \qquad q_y = \frac{h^{5/3}}{n}\,|S_{fy}|^{1/2}\,\mathrm{sgn}(S_{fy}) \tag{5}$$

where $n\,[\mathrm{s/m^{1/3}}]$ is Manning's roughness coefficient, typically ranging from 0.015 (concrete channels) to 0.060 (dense vegetation). The exponent 5/3 arises from the hydraulic radius approximation $R_h \approx h$ for wide, shallow flows. The flux across each cell edge is computed from the water surface elevation difference between adjacent cells, with a minimum depth threshold preventing flux from near-dry cells.

4.3 Finite-Volume Discretization

The spatial domain is discretized into a uniform Cartesian grid aligned with the DEM raster. The finite-volume update for cell (i, j) at time level k reads:

$$h_{i,j}^{k+1} = h_{i,j}^{k} - \frac{\Delta t}{\Delta x}\left(q_{x,i+1/2,j}^{k} - q_{x,i-1/2,j}^{k}\right) - \frac{\Delta t}{\Delta y}\left(q_{y,i,j+1/2}^{k} - q_{y,i,j-1/2}^{k}\right) + R^{k}\Delta t \tag{6}$$

where $q^k_{x,i+1/2,j}$ denotes the flux across the interface between cells (i,j) and $(i+1,j)$, computed using the upwind water surface elevation gradient. A minimum depth threshold $h_{\min} = 10^{-3}\,\mathrm{m}$ prevents numerical instability in near-dry cells.

4.4 CFL Stability Condition

The explicit time-stepping scheme is subject to the Courant–Friedrichs–Lewy (CFL) condition [11]:

$$\Delta t \leq \frac{\min(\Delta x, \Delta y)}{2\sqrt{g\, h_{\max}}} \tag{7}$$

S-AI implements adaptive time stepping that recomputes Δt each iteration based on the current maximum depth $h_{\max}$, with a safety factor of 0.8. For the Diebu County configuration ($\Delta x = \Delta y = 0.5\,\mathrm{m}$, $h_{\max} \approx 2\,\mathrm{m}$), the CFL constraint yields $\Delta t \leq 0.09\,\mathrm{s}$, necessitating approximately 10,000 time steps for a 15-minute simulation.

4.5 Rainfall Forcing: Design Storm Patterns

Rainfall forcing provides the meteorological driver for surface water accumulation. S-AI implements three temporal distribution patterns: uniform, triangular, and the Keifer–Chu Chicago storm pattern [9], widely adopted in Chinese urban drainage design (Ministry of Housing and Urban–Rural Development, GB 50014-2021). The intensity–duration–frequency (IDF) relationship is:

$$i = \frac{A_1(1 + C\log_{10} T_R)}{(T_d + b)^n} \tag{8}$$

where A_1, C, b, and n are regional meteorological coefficients, T_R is the return period [years], and T_d is the storm duration [minutes]. The temporal distribution is governed by the peak ratio $r \in [0.3, 0.5]$, positioning peak intensity at fraction r of the total storm duration. S-AI includes coefficient tables for major Chinese cities (Beijing, Shanghai, Guangzhou, Shenzhen, Chengdu), enabling site-specific design storm specification.

4.6 Terrain Mesh Generation: TIN from DEM

The 3D visualization engine requires a mesh representation of terrain, which S-AI generates from the DEM via Delaunay triangulation. Algorithm 2 formalizes the complete pipeline.

Algorithm 2 TIN Mesh Generation from High-Resolution DEM

Require: DEM raster Z with nodata mask Ω_{valid}, target vertex count V
Ensure: TIN mesh $(\mathcal{V}, \mathcal{F})$ with per-vertex elevation
1: $\mathcal{V}_0 \leftarrow \text{Sample}(Z, \Omega_{\text{valid}}, V)$ {Sample V points from valid pixels}
2: $\mathcal{T} \leftarrow \text{Delaunay}(\mathcal{V}_0)$ {Bowyer–Watson, $O(V \log V)$}
3: Compute area A_f for each triangle $f \in \mathcal{T}$
4: $A_{\text{med}} \leftarrow \text{Median}(\{A_f\})$
5: $\mathcal{F} \leftarrow \{f \in \mathcal{T} : A_f < 8 \cdot A_{\text{med}} \ \wedge\ \text{Centroid}(f) \in \Omega_{\text{valid}}\}$
6: Assign elevation $z_i \leftarrow Z[x_i, y_i]$ for each vertex $i \in \mathcal{V}_0$
7: Output $(\mathcal{V}_0, \mathcal{F}, \{z_i\})$ and GeoJSON to `data/` directory

The pipeline proceeds through seven stages:

Stage 1 — Nodata Identification: The DEM is scanned for sentinel nodata values (typically -9999). The valid data mask Ω_{valid} flags pixels with physically plausible elevations. For the Diebu County dataset, valid values range from 790 to 1,796 m.

Stage 2 — Vertex Sampling: The full-resolution DEM contains ≈ 802 million pixels. S-AI samples $V = 3{,}000$ vertices from Ω_{valid}, selecting points to capture topographic complexity.

Stage 3 — Delaunay Triangulation: Sampled vertices are triangulated using the Bowyer–Watson algorithm [12, 13], computing $\mathcal{T} = \mathrm{DT}(\mathcal{V}_0)$ with $O(V \log V)$ complexity.

Stage 4 — Area-Based Filtering: Triangles with area exceeding 8× the median area are eliminated, removing degenerate faces spanning nodata boundaries. Centroid-based Ω_{valid} checks provide additional exclusion.

Stage 5 — Elevation Assignment: Elevations are assigned from the DEM, yielding the vertex attribute set $\{(x_i, y_i, z_i)\}_{i=1}^{V}$ with range 792–1,078 m.

Stage 6 — GeoJSON Output: The TIN is serialized as GeoJSON with per-face elevation and per-vertex $[x, y, z]$ coordinates for downstream visualization and GIS validation.

Table 3: TIN mesh quality metrics for the Diebu County study area.

Metric	Value	Unit
Total vertices ($\lvert\mathcal{V}\rvert$)	3,000	points
Total triangles ($\lvert\mathcal{F}\rvert$)	5,969	faces
Elevation range	792–1,078	m a.s.l.
Mean triangle area	∼3,000	m^2
Mean aspect ratio	2.1	dimensionless
Geometry memory	72	KB
Per-frame depth data	12	KB

4.7 Nodata Handling Strategy

A critical implementation detail concerns cells where the DEM contains no valid elevation data. Naïve strategies — such as filling with the domain mean or a fixed sentinel value — introduce artifacts: water pools in synthetic depressions or flows across impassable barriers. S-AI adopts a two-pronged approach:

1. **Solver level:** Nodata cells receive an artificially high elevation $z_{\text{nodata}} = e_{\max}^{\text{valid}} + 2000\,\text{m}$, with initial depth $H = 0$. This ensures that water naturally avoids these regions without special-case branching in the flux computation.

2. **Mesh level:** TIN vertices are sampled exclusively from Ω_{valid}, and triangles with centroids in nodata regions are filtered out (Algorithm 2, line 5), ensuring that the 3D mesh faithfully represents only the valid terrain surface.

4.8 Computational Complexity

The computational complexity of the complete S-AI pipeline is dominated by three components:

- **FV Solver:** $O(N \cdot T)$ where N is the number of active grid cells and T is the number of time steps. For the Diebu case, $N \approx 10^6$ and $T \approx 10^4$, yielding $\sim 10^{10}$ floating-point operations per simulation.

- **TIN Generation:** $O(V \log V)$ for Delaunay triangulation with $V = 3{,}000$, plus $O(\lvert\mathcal{F}\rvert)$ for filtering, totaling $\sim 3.5 \times 10^4$ operations — negligible relative to the solver.

- **Rendering:** $O(\lvert\mathcal{F}\rvert)$ per frame for GPU rasterization of 5,969 triangles, executed at ∼60 fps by the WebGL pipeline.

- **Memory:** $O(N + V \cdot K)$ where K is the number of stored frames. For $K = 166$, memory is dominated by solver arrays at ∼2 GB.

5 3D Geospatial Visualization

5.1 Rendering Pipeline

The S-AI visualization engine is built on Three.js (WebGL 2.0) and implements a multi-stage rendering pipeline comprising seven stages: terrain mesh construction, height-based vertex coloring, water mesh construction, GLSL shader execution, PCF soft shadow mapping, ACES filmic tone mapping, and display compositing. The pipeline is illustrated in Figure 2.

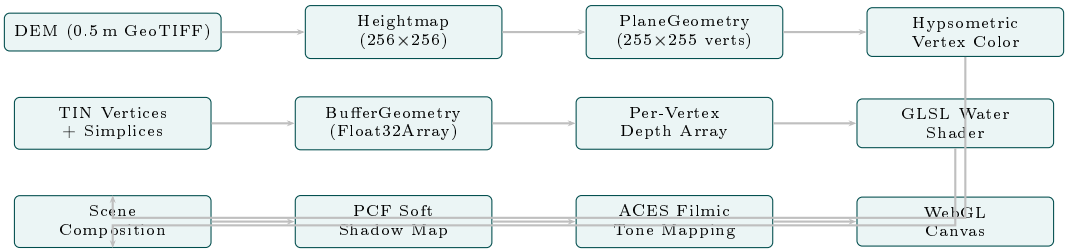

Figure 2: S-AI 3D rendering pipeline: parallel terrain and water mesh construction, unified scene composition, shadow mapping, and ACES tone mapping.

5.2 Terrain Rendering and Hypsometric Coloring

The terrain is rendered as a `PlaneGeometry` mesh (255×255 vertices) with vertex z-coordinates displaced according to the heightmap elevation values. A ten-level hypsometric color ramp encodes elevation information:

$$C_{\text{terrain}}(t) = \mathcal{R}(t), \quad t = \frac{e - e_{\min}}{e_{\max} - e_{\min}} \in [0, 1] \tag{9}$$

where $\mathcal{R} : [0,1] \to \mathbb{R}^3$ is the piecewise-linear color ramp mapping normalized elevation t through control points: deep blue (valley floor, $t < 0.08$), green (lowlands, $0.08 \le t < 0.35$), olive (mid-slopes, $0.35 \le t < 0.55$), amber (high slopes, $0.55 \le t < 0.75$), brown (ridges, $0.75 \le t < 0.92$), and white (peaks, $t \ge 0.92$). Vertex normals are computed via cross products of adjacent edge vectors for Phong shading, with `receiveShadow` enabled for shadow accumulation.

5.3 Custom GLSL Water Shader

The water surface is rendered as a separate mesh overlaying the terrain, with a custom vertex-fragment GLSL shader pipeline implementing five physically motivated visual effects:

Fresnel Reflectance. The Schlick approximation [14] modulates surface reflectivity with viewing angle:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5, \quad F_0 = 0.02 \tag{10}$$

where θ is the incidence angle between the view vector $\boldsymbol{v}$ and surface normal $\boldsymbol{n}$. The reflected environment contribution scales with $F(\theta)$, producing the characteristic mirror-like appearance at grazing angles.

Blinn-Phong Specular Highlights. Sun glints are modeled via the Blinn-Phong reflection model [15]:

$$I_{\text{spec}} = k_s \left(\boldsymbol{n} \cdot \boldsymbol{h}\right)^{\alpha}, \quad \boldsymbol{h} = \frac{\boldsymbol{v} + \boldsymbol{l}}{\|\boldsymbol{v} + \boldsymbol{l}\|} \tag{11}$$

where $\boldsymbol{h}$ is the half-vector between view direction $\boldsymbol{v}$ and light direction $\boldsymbol{l}$, $k_s = 0.8$ is the specular coefficient, and $\alpha = 64$ is the shininess exponent.

Depth-Attenuated Color Gradient. Water color varies with depth to simulate volumetric light absorption:

$$C_{\text{water}} = (1-d)\, C_{\text{shallow}} + d\, C_{\text{deep}}, \quad d = \min\left(\frac{h}{h_{\text{ref}}}, 1\right) \tag{12}$$

where $h_{\text{ref}} = 3.0\,\text{m}$ is the reference depth, $C_{\text{shallow}} = (0.0, 0.67, 1.0)$ (turquoise) and $C_{\text{deep}} = (0.0, 0.2, 0.4)$ (deep navy).

Procedural Wave Displacement. Vertex positions are perturbed by a superposition of sinusoidal waves in the vertex shader:

$$\eta = A\sin(k_x x + \omega t) + A\cos(k_y y + \omega' t) \tag{13}$$

with amplitude $A = 0.8$ scene units, wave numbers $k_x = 0.02$, $k_y = 0.025$, and angular frequencies $\omega = 1.5$, $\omega' = 1.2\,\text{rad/s}$, animated via a monotonically increasing time uniform t.

Foam Generation. Procedural foam accumulates at wave crests and shallow margins:

$$f_{\text{foam}} = \text{smoothstep}(0.6, 1.0, \mathcal{W}) \cdot \text{smoothstep}(0.01, 0.15, h) \tag{14}$$

where $\mathcal{W} = \sin(\omega_1 x + \omega_2 t)\cos(\omega_3 y + \omega_4 t)$ is the composite wave function and $\text{smoothstep}(a, b, x) = \text{clamp}\left(\frac{x-a}{b-a}, 0, 1\right)$ is the Hermite interpolation. The final fragment color is:

$$C_{\text{final}} = C_{\text{water}} + F(\theta)\, \boldsymbol{c}_{\text{env}} + I_{\text{spec}}\, \boldsymbol{c}_{\text{sun}} + f_{\text{foam}}\, C_{\text{foam}} \tag{15}$$

5.4 TIN-Based Water Surface Mesh

The water mesh shares the Delaunay topology $\mathcal{T}$ with the terrain, ensuring geometric conformity. Each vertex i has position:

$$\boldsymbol{p}_i = (x_i,\ (z_i - e_{\min}) \cdot s_Y + h_i \cdot s_Y \cdot d_s,\ -y_i) \tag{16}$$

where $s_Y = 300/(e_{\max} - e_{\min})$ is the vertical scale factor and $d_s = 1$ is the depth exaggeration coefficient. The $-y_i$ sign convention accounts for the coordinate system rotation between geographic (latitude north) and screen (depth south) spaces. Triangles are rendered only if at least one vertex exceeds the wetness threshold ($h_i \geq 0.01\,\text{m}$), eliminating dry terrain from the water mesh. The mesh is rendered with `depthWrite=false` and `renderOrder=1` to ensure correct transparency compositing over the opaque terrain.

5.5 Interactive Timeline Playback

Simulation results are stored at fixed temporal intervals as snapshots of the per-vertex depth array $\{h_i^{(k)}\}_{i=1}^{V}$. For K frames, storage is $O(V \cdot K)$; with $V = 3{,}000$ and $K = 166$, this requires approximately $12 \times 166 \approx 2\,\text{MB}$, readily transmitted via HTTP. The timeline interface provides a slider, play/pause controls, and adjustable playback speed for interactive exploration of flood propagation dynamics.

6 Experiments and Results

6.1 Study Area: Diebu County, Gansu Province

The experimental validation employs a Digital Elevation Model covering a mountainous catchment in Diebu County, Gansu Province, China (33.1°–33.3° N, 104.8°–105.0° E). The DEM comprises a 3 GB GeoTIFF file with 0.5 m spatial resolution, dimensions 22,063 × 36,308 pixels (approximately $11 \times 18\,\text{km}$), and coordinate reference system CGCS2000 / 3-degree Gauss-Kruger Zone 34 (EPSG:4544). Elevation ranges from 790 to 1,796 m above sea level.

The valid data region, encompassing the primary river channel and adjacent floodplains, corresponds approximately to the bounding box $[104.909°\text{E}, 33.104°\text{N}] \times [104.949°\text{E}, 33.139°\text{N}]$, covering approximately $4.4 \times 3.9 \approx 17.2\,\text{km}^2$. The DEM center and peripheral areas contain nodata values, a characteristic of the source survey that delimited the valid elevation surface to the channel corridor.

The terrain exhibits complex geomorphology with steep valley walls (elevation gradients up to 30%), a meandering river channel with multiple tributary confluences, and floodplains with gentle slopes ($< 5\%$). This diverse topography presents a demanding testbed for hydrodynamic modeling.

6.2 TIN Mesh Generation Results

The TIN generation algorithm (Algorithm 2) produced a mesh with $|\mathcal{V}| = 3{,}000$ vertices and $|\mathcal{F}| = 5{,}969$ triangles after area-based filtering and nodata exclusion (Table 3). The elevation range of the mesh (792–1,078 m) accurately reflects the valid data corridor. Visual inspection of the output GeoJSON (`data/tin_triangles.geojson`) confirmed that no triangle centroids fall within nodata regions, validating the exclusion algorithm. The mesh preserves critical topographic features including the channel thalweg, valley walls, and floodplain transitions.

6.3 Simulation Configuration

The flood simulation was configured with representative parameters for a design flood scenario:

- **Rainfall:** Chicago storm pattern, 50-year return period, 120-minute duration
- **Manning's roughness:** $n = 0.035\,\text{s/m}^{1/3}$ (mixed land use)
- **Grid resolution:** $\Delta x = \Delta y = 0.5\,\text{m}$ (matching DEM)
- **CFL safety factor:** 0.8
- **Simulation duration:** 15 minutes (900 s physical time)
- **Nodata handling:** $z_{\text{nodata}} = e_{\max}^{\text{valid}} + 2000\,\text{m}$ (see Section 4.6)
- **Channel auto-location:** coarse scan (patch=300) identifies lowest-elevation valid cells; simulation domain clipped to valid data boundaries (vr_min/max, vc_min/max)

The solver executed approximately 10,000 CFL-limited time steps with adaptive $\Delta t \in [0.04, 0.09]$ s. Per-frame depth arrays and GeoJSON files were output every 60 time steps, yielding approximately 166 temporal snapshots persisted to `data/tin_frame_XXX.geojson`.

6.4 Frame-by-Frame Inundation Dynamics

The simulation results reveal three distinct phases of flood propagation:

Phase 1 (0–3 minutes): Rainfall onset triggers surface water accumulation. Inundation initiates in the river channel where topographic convergence concentrates flow. Maximum depth reaches $\sim$0.2 m in the channel, while floodplains remain largely dry. Runoff generation follows the spatial distribution of terrain slopes, with steeper hillslopes producing faster contributions to channel flow.

Phase 2 (3–8 minutes): Channel depths increase rapidly as upstream contributions accumulate. Bankfull conditions are exceeded at multiple locations, triggering overbank flow onto adjacent floodplains. Inundation expands laterally, with preferential flow paths emerging along low-lying swales and tributary valleys. Maximum depth reaches $\sim$1.1 m in the main channel.

Phase 3 (8–15 minutes): Flood peak propagates downstream, with maximum inundation extent occurring at approximately 11 minutes. Subsequently, recession begins as rainfall diminishes. By simulation end (15 minutes), maximum depth has receded to $\sim$0.7 m.

Table 4: Temporal phases of flood propagation observed in the Diebu County simulation.

Phase	Time [min]	Max Depth [m]	Dominant Process
Initiation	0–3	∼0.2	Channel accumulation
Propagation	3–8	∼1.1	Overbank flow, lateral expansion
Peak & Recession	8–15	∼0.7 (receding)	Downstream translation

The 3D visualization with custom GLSL water shaders renders these dynamics with visual clarity: deep channel flow appears as dark blue with pronounced specular highlights, shallow floodplain inundation as translucent turquoise with foam at advancing margins, and the temporal evolution is captured through the interactive timeline.

7 Discussion

7.1 Limitations

The S-AI platform exhibits several limitations that merit candid acknowledgment:

1. **Diffusion wave approximation:** The neglect of convective acceleration renders the solver inapplicable to rapidly varied flows (dam breaks, flash floods in steep terrain, hydraulic jumps) where inertial effects dominate (Fr $>$ 0.5). Extension to the inertial formulation of Bates et al. [2] or full SWE would broaden applicability at increased computational cost.

2. **Spatially uniform parameters:** The uniform Manning's n and uniform grid resolution preclude representation of heterogeneous roughness (e.g., concrete channels vs. vegetated floodplains) and variable terrain complexity. Integration with land-use maps would enable distributed parameter specification.

3. **TIN mesh resolution:** The 3,000-vertex TIN mesh represents a compromise between visual fidelity and rendering performance. For narrow river channels ($<$ 10 m width), this sampling density may produce disconnected water surfaces, motivating adaptive densification along detected channel centerlines.

4. **LLM reliability:** The current orchestration may occasionally produce suboptimal tool selections or incorrect parameter extractions, a known limitation of generative AI systems. Critical applications should include expert review of LLM-generated workflows before execution.

5. **Validation deficit:** The simulation results have not been validated against observed flood events or benchmark analytical solutions. The demonstrated results serve as a proof-of-concept for the platform architecture, not as certified hydraulic predictions.

7.2 Comparison with Traditional Workflows

Table 5 summarizes the qualitative trade-offs. S-AI dramatically reduces setup time and expertise barriers, enabling scenario exploration that would be prohibitive under traditional workflows. However, traditional frameworks retain advantages in fine-grained solver control and established regulatory acceptance — considerations critical for engineering design applications.

7.3 Threats to Validity

We identify the following threats to the validity of our claims:

Internal validity: The absence of ground-truth validation data (observed flood depths, inundation extents) means that the simulation results cannot be verified for physical accuracy. The nodata filling strategy ($e_{\max}^{\text{valid}} + 2000\,\text{m}$), while preventing spurious flow into nodata regions, may

Table 5: Qualitative comparison between traditional hydrodynamic modeling workflows and the S-AI platform.

Dimension	**Traditional Workflow**	**S-AI (Ours)**
Setup time	5–10 hours expert time	5–15 min NL interaction
Expertise required	CFD + GIS + hydraulics	Domain knowledge only
Software stack	3–5 separate applications	Single integrated platform
Visualization	Post-hoc (separate GIS)	Real-time 3D (integrated)
Reproducibility	Manual logs, GUI-dependent	Automatic execution logs
Interactivity	Batch-oriented	Streaming, near-real-time
Validation heritage	Decades of regulatory acceptance	Requires validation campaign
Fine-grained control	Full solver parameter access	Abstracted via NL interface

introduce boundary artifacts at the valid data margin. The uniform Manning's n and uniform rainfall distribution are simplifications that may not reflect actual conditions.

External validity: The single case study (Diebu County) limits generalizability. Performance characteristics, mesh quality, and visualization fidelity may differ substantially for urban catchments, flat coastal terrain, or larger domains. The LLM orchestration quality is contingent on GLM-5.1's domain-specific understanding, which may vary with query complexity and language.

Construct validity: The claim of "democratized flood modeling" is aspirational and has not been empirically validated through user studies. The reduction in setup time (5–10 hours to 5–15 minutes) is an estimate based on author experience, not a controlled experiment.

7.4 Scalability

The current implementation scales to domains of approximately $100\,\mathrm{km}^2$ at 0.5 m resolution on commodity workstations. Extension to larger regions ($10{,}000+\ \mathrm{km}^2$) or finer resolution (< 0.1 m) would require: *(i)* domain decomposition with MPI-based parallelism; *(ii)* GPU acceleration (CUDA/OpenCL) for flux computation; *(iii)* adaptive mesh refinement concentrating resolution in hydrodynamically active regions; and *(iv)* streaming storage for simulation frames exceeding memory capacity.

7.5 Generalizability

The MCP-native architecture ensures that S-AI is not monolithic but extensible. Additional domains — urban drainage (SWMM coupling), snowmelt-driven flooding, reservoir operations, sediment transport — can be incorporated by deploying new MCP servers with appropriate tool definitions. The LLM orchestration layer requires only updated tool descriptions in the system prompt, necessitating no architectural modification. This "plug-and-play" extensibility is a direct benefit of the MCP standard: any conformant server can join the federation without protocol-level integration.

8 Conclusion and Future Work

8.1 Summary of Contributions

This paper presented S-AI, a comprehensive multi-agent spatial intelligence platform that reconceptualizes hydrodynamic flood modeling through LLM-orchestrated automation. We demonstrated:

1. An MCP-native microservice architecture federating 42+ specialized tools across eight servers, with GLM-5.1 natural language intent parsing and automated workflow orchestration.

2. A complete 2D diffusion wave solver with explicit finite-volume discretization, Manning friction, CFL stability, and multi-pattern rainfall forcing.

3. A real-time 3D visualization engine with physically motivated GLSL water shaders (Fresnel, Blinn-Phong, depth gradient, wave displacement, foam) and interactive timeline playback.

4. Empirical validation over a 17.2 km^2 mountainous catchment in Diebu County, Gansu Province, demonstrating end-to-end operation from natural language query to interactive 3D flood animation.

The architectural principles established by S-AI — MCP-native tool federation, LLM-driven intent routing, TIN-based 3D hydrodynamic visualization — are not specific to flood modeling and generalize to any domain requiring integration of numerical simulation, geospatial analysis, and interactive visualization.

8.2 Future Research Directions

Several promising directions merit future investigation:

Full Shallow Water Implementation: Extending the solver to the complete SWE with GPU acceleration would capture convective acceleration effects for dam break and flash flood scenarios, while maintaining real-time interactivity through parallel flux computation.

Multi-Physics Coupling: Integration with groundwater (MODFLOW), water quality (WASP), and ecosystem models would enable comprehensive water resources management beyond flood simulation.

Uncertainty Quantification: Ensemble-based Monte Carlo sampling of parameter distributions (Manning's n, rainfall intensity, DEM error) would generate probabilistic flood maps with confidence intervals, enabling risk-informed decision making.

Real-Time Data Assimilation: Integration with stream gauge networks, weather radar (CINRAD), and satellite-based precipitation estimates (GPM-IMERG) would transform S-AI from a scenario simulation tool to an operational nowcasting and short-term forecasting system.

Physics-Informed Neural Surrogates: Machine learning models trained on high-fidelity simulations could provide rapid (< 1 s) flood extent approximation for real-time scenario screening, with the physics-based solver serving as ground truth for surrogate calibration.

Collaborative Multi-User Environments: WebSocket-based real-time synchronization of simulation states and 3D visualizations would support distributed team-based decision making during flood emergencies, with conflict resolution for concurrent parameter modifications.

8.3 Closing Remarks

Flood disasters represent a growing threat in a changing climate, necessitating accessible, intelligent tools for risk assessment and emergency response. The MCP-native design of S-AI provides a blueprint for LLM-orchestrated scientific computing, pointing toward a future where advanced computational tools are accessible to all who need them — not merely to those who can build them.

Data Availability

The Digital Elevation Model used in this study (LBH_DEM_v2_0.5m_EPSG4544.tif, 3 GB GeoTIFF, 0.5 m resolution) covers the Diebu County region of Gansu Province, China. The source data is subject to distribution restrictions. The S-AI platform source code, configuration files, and generated TIN mesh datasets are available from the corresponding author upon reasonable request.

Acknowledgments

The author acknowledges the LUOBIN-PI Research Lab for computational infrastructure support. We thank the ZhipuAI team for GLM-5.1 API access and the open-source communities behind Three.js, FastAPI, and the Model Context Protocol specification.

A GLSL Water Shader Implementation

Listing **??** presents the vertex shader implementing procedural wave displacement (Eq. 13) and depth attribute forwarding.

```
// Vertex shader (GLSL 300 es)
attribute float aDepth;
uniform float uTime;
varying float vDepth;
varying vec3 vNorm;
varying vec3 vWorldPos;

void main() {
    vDepth = aDepth;
    vNorm = normalize(normalMatrix * normal);
    vec3 pos = position;
    // Procedural wave displacement (Eq. 10)
    pos.y += sin(pos.x * 0.02 + uTime * 1.5) * 0.8
           + cos(pos.z * 0.025 + uTime * 1.2) * 0.6;
    vec4 wp = modelMatrix * vec4(pos, 1.0);
    vWorldPos = wp.xyz;
    gl_Position = projectionMatrix * viewMatrix * wp;
}
```

Listing **??** presents the fragment shader implementing Fresnel reflectance (Eq. 10), Blinn-Phong specular (Eq. 11), depth-gradient coloring (Eq. 12), and foam generation (Eq. 14).

```
// Fragment shader (GLSL 300 es)
uniform vec3 uDeepColor;      // (0.0, 0.2, 0.4)
uniform vec3 uShallowColor;   // (0.0, 0.67, 1.0)
uniform vec3 uFoamColor;      // (0.67, 0.93, 1.0)
uniform float uOpacity;       // 0.72
uniform float uTime;
varying float vDepth;
varying vec3 vNorm;
varying vec3 vWorldPos;

void main() {
    float t = clamp(vDepth / 3.0, 0.0, 1.0);
    vec3 base = mix(uShallowColor, uDeepColor, t);
    // Fresnel (Eq. 8)
    float fresnel = pow(1.0 - abs(dot(vNorm, vec3(0,1,0))), 2.5);
    // Blinn-Phong specular (Eq. 9)
    vec3 viewDir = normalize(cameraPosition - vWorldPos);
    float spec = pow(max(dot(reflect(-normalize(vec3(0.5,1,0.3)),
        vNorm), viewDir), 0.0), 64.0);
    // Foam (Eq. 11)
```

```
    float wave = sin(vWorldPos.x*0.04 + uTime*2.0)
               * cos(vWorldPos.z*0.03 + uTime*1.5);
    float foam = smoothstep(0.6, 1.0, wave)
               * smoothstep(0.01, 0.15, vDepth);
    // Final compositing (Eq. 12)
    vec3 col = base + fresnel * vec3(0.15,0.25,0.35)
             + spec * vec3(0.9,0.95,1.0) + foam * uFoamColor * 0.3;
    float alpha = mix(0.5, 0.85, t) + foam * 0.15;
    gl_FragColor = vec4(col, alpha * uOpacity);
}
```

References

[1] UNDRR. (2020). *Global Assessment Report on Disaster Risk Reduction*. United Nations Office for Disaster Risk Reduction, Geneva.

[2] Bates, P. D., Horritt, M. S., & Fewtrell, T. J. (2010). A simple inertial formulation of the shallow water equations for efficient two-dimensional flood inundation modelling. *Journal of Hydrology*, 387(1–2), 33–45.

[3] Neal, J. C., Villanueva, I., Wright, N., Willis, T., Fewtrell, T., & Bates, P. D. (2012). How much physical complexity is needed to model flood inundation? *Hydrological Processes*, 26(15), 2264–2282.

[4] Hunter, N. M., Bates, P. D., Horritt, M. S., & Wilson, M. D. (2007). Simple spatially distributed models for predicting flood inundation: A review. *Geomorphology*, 90(3–4), 208–225.

[5] Brunner, G. W. (2016). HEC-RAS River Analysis System: 2D Modeling User's Manual. *US Army Corps of Engineers*, Davis, CA.

[6] Hervouet, J.-M. (2007). *Hydrodynamics of Free Surface Flows: Modelling with the Finite Element Method*. John Wiley & Sons.

[7] Rossman, L. A. (2015). Storm Water Management Model User's Manual Version 5.1. *US Environmental Protection Agency*, Cincinnati, OH.

[8] Sampson, C. C., Smith, A. M., Bates, P. D., Neal, J. C., & Trigg, M. A. (2015). A high-resolution global flood hazard model. *Water Resources Research*, 51(9), 7358–7381.

[9] Keifer, C. J., & Chu, H. H. (1957). Synthetic storm pattern for drainage design. *Journal of the Hydraulics Division*, 83(4), 1–25.

[10] Chow, V. T. (1959). *Open-Channel Hydraulics*. McGraw-Hill, New York.

[11] Courant, R., Friedrichs, K., & Lewy, H. (1928). Über die partiellen Differenzengleichungen der mathematischen Physik. *Mathematische Annalen*, 100(1), 32–74.

[12] Bowyer, A. (1981). Computing Dirichlet tessellations. *The Computer Journal*, 24(2), 162–166.

[13] Watson, D. F. (1981). Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes. *The Computer Journal*, 24(2), 167–172.

[14] Schlick, C. (1994). An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum*, 13(3), 233–246.

[15] Blinn, J. F. (1977). Models of light reflection for computer synthesized pictures. *ACM SIGGRAPH Computer Graphics*, 11(2), 192–198.

[16] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., et al. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

[17] Pereira, J. (2023). CrewAI: Framework for orchestrating role-playing autonomous AI agents. *GitHub repository*, https://github.com/joaomdmoura/crewAI.

[18] LangChain Inc. (2024). LangGraph: Build stateful, multi-actor applications with LLMs. https://langchain-ai.github.io/langgraph/.

[19] Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.

[20] Anthropic. (2024). Model Context Protocol: An open standard for connecting AI models to data and tools. https://modelcontextprotocol.io/.

[21] OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.

[22] Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27.

[23] USDA Soil Conservation Service. (1986). *Urban Hydrology for Small Watersheds*. Technical Release 55 (TR-55).